

# Red Carpenter-Grossberg: patrones, umbrales, rho y clusters

Proyecto: CarpenterGrossberg-TFI-IA

Este apunte resume como la red reconoce patrones de clientes. La idea principal es que la red no trabaja con los numeros originales directamente: primero los convierte en un patron binario de 0 y 1, y luego compara ese patron con prototipos de clusters ya aprendidos.

| Concepto    | Idea corta                                                      |
|-------------|-----------------------------------------------------------------|
| Umbral      | Punto de corte que separa bajo y alto para una variable.        |
| Patron X    | Vector binario del cliente nuevo que queremos clasificar.       |
| Prototipo T | Vector binario que resume lo comun de un cluster.               |
| rho         | Nivel de exigencia para aceptar un patron dentro de un cluster. |
| Cluster     | Grupo de clientes con patrones parecidos.                       |

## 1. Que problema resuelve la red

El objetivo del proyecto es segmentar clientes sin etiquetas previas. La red recibe filas de un CSV, transforma cada fila en un patron binario y va formando grupos segun la similitud entre esos patrones.

La red es incremental: procesa los clientes en orden. Cuando llega un cliente, decide si entra en algun cluster existente o si conviene crear un cluster nuevo.

## 2. Como transforma las variables

La version clasica del algoritmo usada en el proyecto trabaja con patrones binarios, entonces cada variable numerica se transforma en 0 o 1.

Para hacerlo se usa un umbral: un punto de corte. Si el valor del cliente esta por encima o igual al umbral, se escribe 1. Si esta por debajo, se escribe 0.

```
si valor >= umbral -> 1 si valor < umbral -> 0
```

Por defecto, el programa calcula el umbral usando la mediana de cada columna. La mediana es el valor que queda en el medio cuando los datos estan ordenados.

| Valor de monto_total | Comparacion con umbral 900 | Resultado binario |
|----------------------|----------------------------|-------------------|
| 100                  | 100 < 900                  | 0                 |
| 300                  | 300 < 900                  | 0                 |
| 900                  | 900 >= 900                 | 1                 |
| 1200                 | 1200 >= 900                | 1                 |
| 2000                 | 2000 >= 900                | 1                 |

Variables: [frecuencia, monto, promedio, cantidad, recencia] Patron: [1, 1, 0, 1, 0]

## 3. Que es un patron y que es un prototipo

El patron X es el cliente nuevo ya transformado a 0 y 1. El prototipo T es el resumen de un cluster existente.

X = [1, 1, 0, 1, 0] cliente nuevo T = [1, 0, 0, 1, 0] prototipo de un cluster

El prototipo no representa a un cliente puntual. Representa las caracteristicas comunes que fueron quedando en ese cluster. Cuando un cliente entra al cluster, el prototipo se actualiza con una operacion AND.

prototipo actual: [1, 1, 1, 0, 0] patron nuevo: [1, 1, 0, 0, 0] resultado AND: [1, 1, 0, 0, 0]

## 4. Que significa T AND X

La operacion AND compara posicion por posicion. Solo da 1 cuando ambos vectores tienen 1 en la misma posicion.

T = [1, 0, 0, 1, 0] X = [1, 1, 0, 1, 0] T AND X [1, 0, 0, 1, 0]

| Expresion | Significado                                    | Ejemplo |
|-----------|------------------------------------------------|---------|
| T AND X   | Cantidad de coincidencias activas entre T y X. | 2       |
| T         | Cantidad de unos activos en el prototipo.      | 2       |
| X         | Cantidad de unos activos en el cliente nuevo.  | 3       |

## 5. Primera formula: elegir el mejor candidato

$$|T \text{ AND } X| / (0.5 + |T|)$$

Esta formula no decide todavia si el cliente entra al cluster. Sirve para ordenar candidatos: le dice a la red que cluster parece mas prometedor para probar primero.

La formula premia coincidencias activas, pero penaliza un poco a los prototipos con muchas características activas. Asi evita elegir un prototipo demasiado amplio solo porque coincide en muchas cosas.

| Cluster | T AND X | T | Calculo         | Puntaje |
|---------|---------|---|-----------------|---------|
| T1      | 2       | 2 | $2 / (0.5 + 2)$ | 0.80    |
| T2      | 3       | 4 | $3 / (0.5 + 4)$ | 0.67    |

El 0.5 es una constante de normalizacion. Suaviza el puntaje y evita resultados demasiado extremos o empates problematicos.

## 6. Segunda formula: prueba de vigilancia

$$|T \text{ AND } X| / |X| \geq \rho$$

Esta formula decide si el cluster acepta al cliente. Pregunta: de todas las características activas del cliente nuevo, cuantas reconoce el cluster?

$$X = [1, 1, 0, 1, 0] \rightarrow |X| = 3 \quad T = [1, 0, 0, 1, 0] \rightarrow |T \text{ AND } X| = 2 \quad 2 / 3 = 0.66$$

| rho  | Resultado                  | Interpretacion                 |
|------|----------------------------|--------------------------------|
| 0.6  | $0.66 \geq 0.6$ : acepta   | La red es mas flexible.        |
| 0.8  | $0.66 \geq 0.8$ : rechaza  | La red es mas exigente.        |
| 0.95 | $0.66 \geq 0.95$ : rechaza | Exige casi coincidencia total. |

## 7. Que es rho

rho es el parametro de vigilancia: el nivel minimo de parecido que se exige para que un cliente entre en un cluster existente.

- rho bajo: acepta patrones mas distintos y suele crear menos clusters.
- rho alto: exige patrones mas parecidos y suele crear mas clusters.
- rho = 0.8, usado en las demos, es un valor bastante exigente pero no extremo.

## 8. Que son los clusters

Un cluster es un grupo de clientes con patrones parecidos. La red no conoce nombres comerciales al principio; crea grupos numerados.

| Etiqueta estimada                              | Lectura posible                                                  |
|------------------------------------------------|------------------------------------------------------------------|
| alto_monto_total_alto_monto_promedio           | Clientes con gasto total y gasto promedio altos.                 |
| bajo_monto_total_alto_dias_desde_ultima_compra | Clientes de bajo gasto y mucha distancia desde la ultima compra. |
| alto_monto_compra_bajo_edad                    | Clientes jovenes con monto de compra alto.                       |

Estas etiquetas no son verdades absolutas. Son ayudas de interpretacion. La red matematicamente solo devuelve clusters.

## 9. Proceso completo de reconocimiento

| Paso | Que hace la red                                                     |
|------|---------------------------------------------------------------------|
| 1    | Convierte valores numericos a 0 y 1 usando umbrales.                |
| 2    | Obtiene el patron binario X del cliente.                            |
| 3    | Compara X con cada prototipo T usando T AND X.                      |
| 4    | Usa $ T \text{ AND } X  / (0.5 +  T )$ para elegir candidato.       |
| 5    | Aplica $ T \text{ AND } X  /  X  \geq \rho$ para decidir si acepta. |
| 6    | Si acepta, actualiza el prototipo con AND.                          |
| 7    | Si no acepta, prueba el siguiente candidato.                        |
| 8    | Si ningun cluster acepta, crea un cluster nuevo.                    |

## 10. Ejemplo completo

$X = [1, 1, 0, 1, 0]$   $T1 = [1, 0, 0, 1, 0]$   $T2 = [0, 1, 0, 0, 1]$   $T3 = [1, 1, 1, 1, 0]$

Primero se calcula el puntaje de eleccion para ordenar candidatos. Despues se aplica la vigilancia sobre el mejor candidato. Si no pasa rho, se prueba el siguiente mejor.

| Formula                              | Pregunta que responde                  | Uso                |
|--------------------------------------|----------------------------------------|--------------------|
| $ T \text{ AND } X  / (0.5 +  T )$   | Que cluster parece el mejor candidato? | Ordena candidatos. |
| $ T \text{ AND } X  /  X  \geq \rho$ | El parecido alcanza el nivel exigido?  | Acepta o rechaza.  |

## 11. Relacion con el codigo del proyecto

| Parte del codigo       | Responsabilidad                                                      |
|------------------------|----------------------------------------------------------------------|
| binarize_rows          | Calcula umbrales y transforma filas numericas en patrones 0/1.       |
| _matching_score        | Implementa $ T \text{ AND } X  / (0.5 +  T )$ .                      |
| _vigilance_similarity  | Implementa $ T \text{ AND } X  /  X $ .                              |
| predict_one            | Elige candidato, aplica vigilancia, asigna cluster o crea uno nuevo. |
| build_cluster_profiles | Calcula promedios por cluster y genera etiquetas interpretativas.    |

## 12. Resumen para recordar

La red reconoce patrones comparando vectores binarios. Primero convierte cada variable numerica en bajo o alto segun un umbral. Luego compara el patron del cliente con prototipos de clusters y decide si alguno se parece lo suficiente segun rho.

La primera formula elige el candidato mas prometedor. La segunda formula decide si el parecido alcanza. Los clusters son grupos de clientes parecidos, y rho regula que tan estricta es la red.

| Termino | Definicion                                                          |
|---------|---------------------------------------------------------------------|
| Umbral  | Punto de corte usado para convertir una variable numerica en 0 o 1. |

| Termino   | Definicion                                                   |
|-----------|--------------------------------------------------------------|
| Patron    | Vector binario que representa a un cliente.                  |
| Prototipo | Vector binario que resume lo comun de un cluster.            |
| AND       | Operacion que conserva 1 solo donde ambos vectores tienen 1. |
| rho       | Exigencia minima de parecido para aceptar un patron.         |
| Cluster   | Grupo de clientes con patrones similares.                    |

## 13. Los prototipos ya vienen por defecto?

No. La red empieza vacia: al inicio no tiene clusters ni prototipos cargados. En el codigo esto aparece como una lista vacia.

```
self.prototypes = []
```

El primer patron que llega crea el primer cluster. Como no hay nada con que compararlo, ese mismo patron se copia como prototipo inicial.

```
primer cliente: X = [1, 1, 0, 1, 0] como no hay prototipos: Cluster 1 = [1, 1, 0, 1, 0]
```

Si algun prototipo supera la prueba de vigilancia, el cliente entra en ese cluster y el prototipo se actualiza con AND. Si ninguno lo acepta, se crea un cluster nuevo.

```
prototipo actual: T = [1, 1, 1, 0, 0] cliente nuevo: X = [1, 1, 0, 0, 0] nuevo prototipo: [1, 1, 0, 0, 0]
```

Entonces los prototipos se aprenden durante la ejecucion. No son etiquetas previas ni grupos definidos a mano.

## 14. Transformacion de los datasets y variables usadas

Esta seccion separa tres niveles: columnas originales del dataset, columnas procesadas guardadas en el proyecto y variables que finalmente entraron a la red.

Nota: en el repositorio no esta el script exacto de preprocesamiento original. Por eso, la transformacion se documenta a partir de los CSV procesados presentes en datasets/ y de las columnas originales vistas en las capturas.

CSV original -> limpieza / codificacion / resumen -> CSV procesado -> seleccion de variables -> binarizacion 0/1 -> red

### 14.1 Dataset Online Retail

El dataset original de Online Retail esta a nivel transaccion/producto: una fila representa una linea de factura, no un cliente completo. La red necesita una fila por cliente, por eso se resumieron transacciones en metricas por cliente.

| Columna original | Columna procesada o derivada | Usada? | Como se transformo / uso                                      |
|------------------|------------------------------|--------|---------------------------------------------------------------|
| InvoiceNo        | frecuencia_compra            | Si     | Se cuenta la cantidad de facturas/compras por cliente.        |
| StockCode        | No usada directamente        | No     | Identifica productos; no entro como variable final.           |
| Description      | No usada directamente        | No     | Nombre del producto; texto categorico descartado.             |
| Quantity         | cantidad_productos           | Si     | Se suma la cantidad comprada por cliente.                     |
| InvoiceDate      | dias_desde_ultima_compra     | Si     | Se toma la ultima compra y se calcula la recencia en dias.    |
| UnitPrice        | monto_total / monto_promedio | Si     | Se combina con Quantity para calcular gasto total y promedio. |
| CustomerID       | id_cliente                   | No     | Se usa solo como identificador, no para segmentar.            |
| Country          | No usada directamente        | No     | Pais del cliente; no entro en la corrida principal.           |

Variables finales usadas por la red en online\_retail\_procesado.csv:

frecuencia\_compra, monto\_total, monto\_promedio, cantidad\_productos, dias\_desde\_ultima\_compra

Columnas presentes en el CSV procesado pero no usadas como entrada:

id\_cliente, perfil\_referencia, uso\_descuentos, variedad\_productos

Ejemplo: UnitPrice y Quantity no aparecen con esos nombres porque se transformaron en metricas de gasto como monto\_total y monto\_promedio.

## 14.2 Dataset Customer Shopping Behavior

Este dataset trae muchas columnas categoricas. Para que la red pueda trabajar, algunas se conservaron como numeros, otras se codificaron y otras se descartaron.

| Columna original       | Columna procesada o derivada  | Usada?     | Como se transformo / uso                                  |
|------------------------|-------------------------------|------------|-----------------------------------------------------------|
| Customer ID            | id_cliente                    | No         | Se renombro como identificador.                           |
| Age                    | edad                          | Si         | Se mantuvo como variable numerica.                        |
| Gender                 | No incluida                   | No         | Se descarto en esta version procesada.                    |
| Item Purchased         | No incluida                   | No         | Producto especifico; texto/categoria descartada.          |
| Category               | categoria_producto_codificada | Si         | Se convirtio a codigo numerico.                           |
| Purchase Amount (USD)  | monto_compra                  | Si         | Se renombro y se mantuvo como monto numerico.             |
| Location               | No incluida                   | No         | Se descarto.                                              |
| Size                   | No incluida                   | No         | Se descarto.                                              |
| Color                  | No incluida                   | No         | Se descarto.                                              |
| Season                 | temporada_codificada          | No         | Se codifico, pero no se uso en la corrida principal.      |
| Review Rating          | No incluida                   | No         | Se descarto.                                              |
| Subscription Status    | No incluida                   | No         | Se descarto.                                              |
| Shipping Type          | No incluida                   | No         | Se descarto.                                              |
| Discount Applied       | uso_descuentos                | Si         | Se transformo a 0/1 segun uso de descuento.               |
| Promo Code Used        | No aparece separada           | No directo | Puede haberse descartado o resumido junto con descuentos. |
| Previous Purchases     | frecuencia_compra             | Si         | Se adapto como frecuencia numerica de compra.             |
| Payment Method         | metodo_pago_codificado        | No         | Se codifico, pero no se uso en la corrida principal.      |
| Frequency of Purchases | frecuencia_compra             | Si         | Se adapto a valor/codigo numerico de frecuencia.          |

Variables finales usadas por la red en customer\_shopping\_behavior\_procesado.csv:

edad, monto\_compra, uso\_descuentos, frecuencia\_compra, categoria\_producto\_codificada

Variables adaptadas pero no usadas en la corrida principal:

metodo\_pago\_codificado, temporada\_codificada

Despues de estas adaptaciones, la red todavia hace un paso mas: binariza las variables numericas usando umbrales. Por ejemplo, monto\_compra no entra como 252 sino como 1 si esta por encima del umbral o 0 si esta por debajo.

## 15. Como se combinaron las columnas originales para calcular las nuevas

Esta seccion baja a formulas concretas la idea de preprocesamiento. En especial, Online Retail necesitaba agregarse por cliente porque el original viene por linea de factura. En Customer Shopping Behavior, la transformacion principal fue renombrar, seleccionar y codificar categorias.

Aclaracion importante: el repo contiene los CSV ya procesados, pero no incluye el script historico que genero esos CSV desde Kaggle. Por eso estas formulas documentan el criterio de transformacion consistente con las columnas finales del proyecto.

### 15.1 Online Retail: de lineas de factura a metricas por cliente

En el dataset original, cada fila puede verse como una linea de venta: factura, producto, cantidad, fecha, precio unitario y cliente. Para que la red segmente clientes, primero se agrupan todas las filas por CustomerID.

Agrupar por CustomerID Para cada cliente, calcular metricas resumidas usando sus facturas y productos.

| Nueva columna            | Columnas originales usadas                       | Formula / criterio de calculo                                                                                                                                                                                                                |
|--------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id_cliente               | CustomerID                                       | Se copia o renombra el identificador del cliente. No entra como feature de la red.                                                                                                                                                           |
| frecuencia_compra        | CustomerID + InvoiceNo                           | Cantidad de facturas/compras distintas del cliente: <code>count_distinct(InvoiceNo)</code> . Si se limpian cancelaciones, se excluyen InvoiceNo que empiezan con C.                                                                          |
| cantidad_productos       | CustomerID + Quantity                            | Suma de productos comprados por cliente: <code>sum(Quantity)</code> .                                                                                                                                                                        |
| monto_total              | CustomerID + Quantity + UnitPrice                | Primero se calcula <code>total_linea = Quantity * UnitPrice</code> . Luego <code>monto_total = sum(total_linea)</code> por cliente.                                                                                                          |
| monto_promedio           | monto_total + frecuencia_compra                  | Promedio de gasto por compra: <code>monto_total / frecuencia_compra</code> . Otra variante posible es <code>avg(total_linea)</code> , pero en este proyecto la idea documentada es promedio por compra.                                      |
| dias_desde_ultima_compra | CustomerID + InvoiceDate                         | Se toma <code>ultima_compra = max(InvoiceDate)</code> por cliente. Luego <code>dias_desde_ultima_compra = fecha_referencia - ultima_compra</code> .                                                                                          |
| uso_descuentos           | No hay columna directa en Online Retail original | En el CSV procesado esta disponible, pero no se uso como entrada principal. Si se calcula desde retail original, requiere una regla externa, por ejemplo detectar promociones/descuentos; no sale directo de las columnas basicas mostradas. |
| variedad_productos       | CustomerID + StockCode                           | Cantidad de productos distintos comprados: <code>count_distinct(StockCode)</code> . Esta columna existe en el procesado, pero no se uso en la corrida principal.                                                                             |

Ejemplo conceptual con dos lineas del mismo cliente:

Cliente 1001 compra: Linea 1: Quantity=2, UnitPrice=10 -> total\_linea=20 Linea 2: Quantity=3, UnitPrice=15 -> total\_linea=45   
monto\_total = 20 + 45 = 65   
cantidad\_productos = 2 + 3 = 5   
frecuencia\_compra = cantidad de facturas distintas   
monto\_promedio = monto\_total / frecuencia\_compra



## 15.2 Customer Shopping Behavior: seleccion, renombrado y codificacion

En este dataset, varias columnas ya describen compras/clientes en una forma mas directa. Por eso no hizo falta resumir facturas como en Online Retail. El trabajo principal fue seleccionar variables utiles, renombrarlas y convertir categorias textuales a numeros.

| Nueva columna                 | Columnas originales usadas                        | Formula / criterio de calculo                                                                                                                                                                     |
|-------------------------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id_cliente                    | Customer ID                                       | Se copia o renombra el identificador. No entra como feature de la red.                                                                                                                            |
| edad                          | Age                                               | Se copia como numero.                                                                                                                                                                             |
| monto_compra                  | Purchase Amount (USD)                             | Se copia/renombra como monto numerico de compra.                                                                                                                                                  |
| uso_descuentos                | Discount Applied                                  | Se convierte a binario. Ejemplo: Yes -> 1, No -> 0. Si se combina con Promo Code Used, puede marcar 1 cuando hubo descuento o codigo promocional.                                                 |
| frecuencia_compra             | Frequency of Purchases y/o Previous Purchases     | Se transforma a numero. Si Frequency of Purchases es textual, se codifica por nivel; Previous Purchases puede usarse como recuento historico. En el CSV procesado final quedo como valores 0 a 8. |
| categoria_producto_codificada | Category                                          | Se convierte cada categoria a un codigo numerico. Ejemplo conceptual: Clothing -> 1, Footwear -> 2, Accessories -> 3. El diccionario exacto no esta en el repo.                                   |
| metodo_pago_codificado        | Payment Method                                    | Se convierte cada metodo de pago a codigo numerico. Quedo disponible pero no se uso en la corrida principal.                                                                                      |
| temporada_codificada          | Season                                            | Se convierte cada temporada a codigo numerico. Quedo disponible pero no se uso en la corrida principal.                                                                                           |
| perfil_referencia             | Criterio de interpretacion armado para la entrega | Etiqueta auxiliar para comparar/interpretar perfiles. No entra a la red.                                                                                                                          |

Ejemplo conceptual de codificacion:

```
Discount Applied: Yes -> 1 No -> 0 Category: Clothing -> 1 Footwear -> 2 Accessories -> 3 Outerwear -> 4
Luego la red vuelve a binarizar: si categoria_producto_codificada >= umbral -> 1 si categoria_producto_codificada < umbral -> 0
```

Esta doble transformacion puede sonar rara, pero son pasos distintos: primero se convierte texto a numero para poder procesarlo; despues la red convierte ese numero a 0/1 usando el umbral.

## 16. Como se codificaron las variables categoricas

Codificar significa convertir una categoria escrita en texto a un valor numerico. Esto es necesario porque el algoritmo del proyecto solo puede trabajar con numeros y, despues, con patrones binarios de 0 y 1.

Hay dos pasos distintos que no conviene mezclar: primero se codifica texto a numero; despues la red binariza ese numero usando un umbral.

```
texto categorico -> codigo numerico -> comparacion con umbral -> 0 o 1 Ejemplo conceptual: Category = Clothing -> categoria_producto_codificada = 1 Si el umbral de categoria_producto_codificada es 3: 1 < 3 -> patron binario = 0
```

### 16.1 Codificaciones visibles en los CSV procesados

| Dataset                    | Columna procesada             | Valores visibles | Significado                                                                                                     |
|----------------------------|-------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------|
| customer_shopping_behavior | uso_descuentos                | 0, 1             | Variable binaria: 0 indica que no usa descuento; 1 indica que usa descuento.                                    |
| customer_shopping_behavior | categoria_producto_codificada | 1, 2, 3, 4, 5    | Categoria de producto convertida a codigo numerico. El diccionario exacto categoria->numero no esta en el repo. |
| customer_shopping_behavior | metodo_pago_codificado        | 1, 2, 3          | Metodo de pago convertido a codigo numerico. Disponible, pero no usado en la corrida principal.                 |
| customer_shopping_behavior | temporada_codificada          | 1, 2, 3, 4       | Temporada convertida a codigo numerico. Disponible, pero no usada en la corrida principal.                      |
| customer_shopping_behavior | frecuencia_compra             | 0 a 8            | Frecuencia adaptada a escala numerica/codigo. Se uso como variable de entrada.                                  |
| online_retail              | uso_descuentos                | 0, 1             | Variable disponible en el CSV procesado, pero no usada como entrada principal.                                  |

### 16.2 Ejemplo de diccionario de codificacion

El diccionario exacto usado para cada categoria no aparece guardado en el repositorio. Si se reconstruyera el preprocesamiento, se podria usar un diccionario fijo como este:

```
Category: Clothing -> 1 Footwear -> 2 Accessories -> 3 Outerwear -> 4 Jewelry -> 5 Payment Method: Cash -> 1 Credit Card -> 2 PayPal -> 3 Season: Spring -> 1 Summer -> 2 Fall -> 3 Winter -> 4
```

Lo importante no es el numero elegido en si, sino que el mismo texto siempre reciba el mismo codigo. Por ejemplo, si Clothing se codifica como 1, todas las filas Clothing deben quedar como 1.

### 16.3 De codigo numerico a patron binario

Una vez que la categoria ya es numero, la red no la usa todavia como categoria pura. La compara con el umbral calculado por mediana y la convierte a 0 o 1.

```
Supongamos: categoria_producto_codificada = 4 umbral de categoria_producto_codificada = 3 4 >= 3 -> 1 Si otra fila tiene categoria_producto_codificada = 2: 2 < 3 -> 0
```

Por eso una variable categorica pasa por dos transformaciones: primero se vuelve codigo numerico y luego se vuelve bit binario para el patron que reconoce la red.

## 17. Que pasa si una codificacion tiene mas valores que 0 y 1

Cuando una categoria se codifica con varios numeros, por ejemplo 1 a 5, todavia no esta lista para la red clasica. La red Carpenter-Grossberg usada en el proyecto necesita patrones binarios, entonces ese codigo numerico se vuelve a convertir en 0 o 1 mediante un umbral.

```
Category: Clothing -> 1 Footwear -> 2 Accessories -> 3 Outerwear -> 4 Jewelry -> 5 Umbral = 3 1 < 3 -> 0 2 < 3 -> 0 3 >= 3 -> 1 4 >= 3 -> 1 5 >= 3 -> 1
```

| Categoria   | Codigo | Despues del umbral |
|-------------|--------|--------------------|
| Clothing    | 1      | 0                  |
| Footwear    | 2      | 0                  |
| Accessories | 3      | 1                  |
| Outerwear   | 4      | 1                  |
| Jewelry     | 5      | 1                  |

Esto funciona tecnicamente porque la red recibe 0/1, pero pierde detalle: las cinco categorias terminan reducidas a dos grupos, codigo bajo y codigo alto.

La limitacion principal es que los codigos pueden crear un orden artificial. Por ejemplo, si Jewelry = 5 y Clothing = 1, la red puede tratar Jewelry como mas alto que Clothing, aunque en realidad son categorias distintas y no cantidades ordenadas.

### 17.1 Alternativa: codificacion one-hot

Para categorias nominales, una alternativa mas fiel es one-hot. En vez de guardar una sola columna con valores 1 a 5, se crea una columna binaria por categoria.

```
Category = Footwear Clothing = 0 Footwear = 1 Accessories = 0 Outerwear = 0 Jewelry = 0
```

| Enfoque                  | Ventaja                                                  | Desventaja                                     |
|--------------------------|----------------------------------------------------------|------------------------------------------------|
| Codigo numerico + umbral | Simple y compacto. Cumple rapido con la entrada binaria. | Puede perder detalle y crear orden artificial. |
| One-hot                  | Conserva mejor categorias sin orden.                     | Aumenta la cantidad de variables del patron.   |

Para defender el proyecto: las categorias se codificaron numericamente para poder procesarlas y luego se binarizaron con umbrales porque la red clasica trabaja con patrones binarios. Es una simplificacion valida para una implementacion didactica, aunque one-hot seria mas adecuado si se quisiera preservar mejor las categorias nominales.